

HASHIRU

WHITEPAPER • NOVEMBER 2026

Agents That Hold the Money

HASHIRU v2 builds teams of language-model agents that hire, spend and answer within the money they are given

THE SYSTEM

An economy for agents

Each agent holds a pool reserved from its hirer's and hires models and priced agents over A2A and MCP.

THE GUARDS

Spending bounded by construction

Each call is bounded by what its agent holds (as closely as the hub counts tokens), firing refunds what was not spent, and an agent with one step left must answer.

THE RESULT

Guards that hold

Over 551 live hires, none spent more than 1.11 times its reservation, against 372 times as first built.

Kunal Pai
UCLA
kunpai@ucla.edu

Harshil Patel
UC Davis
hpppatel@ucdavis.edu

HASHIRU v2 is source-available under the PolyForm Noncommercial License 1.0.0 at https://github.com/HASHIRU-AI/HASHIRU

Executive Summary

A team of language-model agents spends money every time it hires a strong cloud model for a hard step, a cheap one for an easy step, or an outside agent that sells an exact answer. HASHIRU v2 gives that money to the agents themselves. Each agent holds a pool reserved from its hirer's and spends it by hiring, over the open protocols agents and tools now share (A2A and MCP). The hub makes that spending safe. A call is bounded by what its agent holds before it is made, firing returns what was not spent, and an agent with one step left is made to answer.

What HASHIRU v2 delivers

- **Guards that hold.** Over 551 hires in 120 live runs, none spent more than 1.11 times its reservation (13 went over by 9 to 11%, where the hub's token count ran low on long inputs). As first built, HASHIRU let a hire spend 372 times its reservation and lost half its runs.
- **Long inputs kept out of the CEO's context.** Holding long sections back raised the agents' score from 0.463 to 0.599 of the value (a paired $+0.136$, 95% interval $[+0.067, +0.216]$) and cut the CEO's own calls from 43% of the budget to 32%.
- **Where the agents win, and what holds them back.** Where the budget pays for the CEO, the agents beat the cheap model alone on hard jobs a specialist can do (79% against 69% right) and match it overall (0.737 against 0.740), but fund the dear model below its price (55% of their spend, for 12 right answers in 79).

Where it stands, and where it goes

These are small runs ($33.21 in all) on outcome tables that fail three of seven validity checks (§4.6). Replayed at budgets up to twice what the best answers cost, a solver that adapts during the task rises to 0.88 while the cheap model alone stays near 0.75, and the agents ran live only at the tighter budget. HASHIRU v2 is a step toward agent economies in which agents hold budgets, buy from one another and account for every dollar.

Contents

1 Introduction 2
2 Who Splits the Money Today 4
3 An Economy for Agents 5
4 How We Measured 9
5 Results 12
6 Limitations and Scope 18
7 Toward Economies of Agents 19

1

Introduction

Teams of language-model agents cost money, and how a team's budget is split decides what the team can do. A hierarchical system (a CEO agent that hires managers and workers, as in HASHIRU v1 [1], or the orchestrators of AutoGen [2], MetaGPT [3] and Magentic-One [4]) spends on every hire, and the hires differ in price (a strong cloud model for a hard step, a cheap one for an easy step, an outside agent that sells an exact answer). A team that spends its budget on the wrong steps fails on the ones that needed it, and a team that cannot bound its spending cannot be trusted with a budget at all.

Agent software has started to hand this money to the agents. Automaton's `fund_child` transfers credits to a child agent [5]; wayland-core's `delegate` gives a sub-agent a dollar ceiling charged to it and its descendants [6]; AgentLance lets a manager model set budgets for subtasks within its payment [7]. Budget-aware systems that are evaluated split the money with an optimizer outside the agents instead (BAMAS [8], ZEBRA [9]), and benchmarks of budget awareness (BAGEN [10], R^3-Bench [11], EcoAgent-Bench [12]) measure one agent spending its own tokens or actions. A ceiling that is checked after a call crosses it does not keep the call from crossing it, and none of these systems is measured against the best split its money could buy. No arrangement covers the case where agents hold the money for a whole hiring tree, every call is bounded by what its agent holds before it is made, and the resulting allocation is measured against a known optimum.

To address these limitations, we built HASHIRU v2. Every agent is its own A2A service, every tool is served over MCP, and every hire reserves money from its hirer's pool. The hub, not the prompt, keeps spending safe. A call's output is capped to what its agent has left, a call that cannot pay for an answer is refused with its price, firing returns what was not spent, and an agent with one step left answers with what it has. Because every reservation, call and refund is in the trace, the allocation can be measured, and we measured it on a benchmark built so that the best split of the money is known.

What this whitepaper contributes

- **HASHIRU v2, an economy for agents.** Each agent holds a pool reserved from its hirer's, in dollars and, for local models, measured memory; it declares how hard a job is and the hub hires the cheapest model that fits, or it buys from a priced remote agent or a whole remote team over A2A (§3).
- **Spending bounded by construction.** Pools are reserved and refunded, calls are bounded by their pool before they are made (as closely as the hub counts their tokens), and an agent with one step left gets a closing turn in which it must answer (§3.2).
- **Guards that hold, and a measured account of where agents win.** Over 551 live hires, none spent more than 1.11 times its reservation, where HASHIRU as first built let one spend 372 times. On projects whose best split is known, holding long inputs out of the CEO's context raised the agents' score by 0.136. Where the budget pays for the CEO, the agents beat the cheap model alone on hard jobs a specialist can do (79% against 69% right) and match it overall, and replays at four budgets show where an allocator that adapts pulls ahead (§5).

The guards earn their place in the measurements. Run as first built, the same agents overran a reservation by up to 372 times and lost half their runs to an empty pool; with the call bound and the closing turn, no hire overran by more than the error of the hub's token count. The agents' allocation is the open problem, and the trace turned it into engineering. It showed the CEO paying to reread every long document on every turn, and holding those documents back raised the agents' score by 0.136. The trace also shows where the agents win and why they do not win more. They send hard grids to an exact solver and retry hard jobs on dearer models, which puts them ahead of the cheap model on hard jobs that a specialist can do. But they fund the dear model below its price, so its hires took 55% of what they spent and returned 12 right answers in 79. Replays of the same projects at other budgets show what is at stake. As the budget grows eightfold (from a quarter of what the best answers cost to twice that), the cheap model alone stays near 0.75 of the value, while a solver that adapts during the task as the agents do rises from 0.70 to 0.88. What remains is located as precisely, in the dear model's funding, in a budget floor below which a CEO that pays for each turn cannot work, and in the jobs it answers itself rather than handing off. Each of these is where HASHIRU goes next (§7).

2

Who Splits the Money Today

We organize prior work by who decides how the money is split, and when (Table 1).

An optimizer outside the agents. BAMAS [8] picks the models with an integer program and the collaboration topology with reinforcement learning, before the task. MasRouter [13] routes each query through a learned controller. ATOM [14] predicts a query's difficulty and sets a collaboration budget; MALBO [15] and SC-MAS [16] optimize team composition. ZEBRA [9] splits dollars across the phases of a fixed pipeline with a Lagrangian solver before execution, and its language-model baseline loses to a knapsack solver. Mixture-of-Models [17] runs preset team profiles. In each, the agents that do the work do not hold the money. PURSE includes this arrangement as a baseline (a knapsack over calibrated statistics, §4.5).

Routing and cascades. Model routers pick one model per query. RouteLLM [18] learns from preference data, Hybrid LLM [19] routes between a small and a large model by predicted difficulty, and FrugalGPT [20] cascades from cheap to expensive models until an answer is accepted. RouterBench [21] evaluates routers on precomputed outputs against an oracle router, the evaluation method PURSE's replay follows. A router prices one query at a time; PURSE prices a project whose jobs share one budget, so that spending on one job takes money from another.

Agents that fund agents. Agent runtimes released in 2026 let a model fund the agents it spawns. Automaton transfers credits to a child that runs the same runtime [5], and wayland-core's `delegate` tool gives a sub-agent a dollar ceiling within the parent's remaining envelope, charged to the sub-agent and its descendants [6]. AgentLance [7] lets a manager model set enforced budgets for subtasks within its payment, one level deep, with a market allocator choosing the workers, and evaluates the whole system on benchmarks. HASHIRU v1 [1] preceded these with a CEO that hires and fires agents under cost and memory budgets, one level deep. None of these evaluates how the agents divide the money.

Budget awareness of one agent. BATS [22] makes a tool-using agent aware of its remaining budget; BAGEN [10] finds that strong agents are not necessarily budget-aware and are over-optimistic about tasks unlikely to succeed with the budget left. R^3-Bench [11] has one model split its own output tokens or actions across six problems of hidden difficulty and scores it against an offline knapsack oracle built from single-problem response curves; PURSE builds its oracle the same way. EcoAgent-Bench [12] prices an agent's actions (lookups, searches, a stronger model, a human), AllocBench [23] measures investment in reusable tools, and CostBench [24] measures cost-optimal planning and replanning when tools fail or prices change. In all of them one agent spends on itself.

Local and cloud models, and agent economies. Minions [25] and PAPILLON [26] pair a local model with a cloud model under a fixed protocol. Virtual agent economies [27] and Magentic Marketplace [28] study markets between agents; payment protocols (AP2 [29], the A2A x402 extension [30]) settle

payments but do not allocate. HASHIRU v2 speaks the open protocols these build on, MCP for tools [31] and A2A for agents [32].

Table 1: Who splits the money, and when. HASHIRU v2 shares agent-held budgets with 2026 agent runtimes; it bounds each call before it is made and is measured against a known optimum.

Line of work	Who allocates	When	Evaluated against an optimum
BAMAS, ZEBRA, MasRouter	optimizer	before the task	no
RouteLLM, FrugalGPT	router, cascade	per query	router oracle
Automaton, wayland-core	the model	during	no
AgentLance	manager model	per subtask	no
R^3-Bench, EcoAgent	one agent, on itself	during	yes
HASHIRU v2	**the agents, to depth 2**	**during, each call bounded**	**yes**

HASHIRU v2 shares agent-held budgets with these runtimes and the oracle construction with R^3-Bench, but it is different in two dimensions. (1) *Bounded before the call.* A hire reserves money (and memory) that firing refunds, and each call's output is capped to what its agent holds before the call is made, where wayland-core's sub-agent stops itself once it has crossed its cap. (2) *Measured against an optimum.* To our knowledge, it is the first system whose agent-held budgets are evaluated across a hiring tree against solver, up-front and fixed-team splits at equal dollars.

3

An Economy for Agents

HASHIRU v2 rebuilds v1 [1] on open protocols. A user's request goes to a CEO agent, which may hire *managers* (agents that use tools and may hire) and *workers* (agents that answer once), to a configurable depth (two levels below the CEO by default). Every agent is its own A2A service with its own Agent Card, every tool is served over MCP, and HASHIRU itself is reachable as an OpenAI-compatible model, an A2A agent and an MCP server. One process serves everything on one port. Every model call, tool call and message is written to a JSONL trace. Figure 1 shows one hiring tree.

3.1 From v1 to v2

v1 had one level of hierarchy, a hand-written table of resource units, and no analysis of the local-against-cloud trade (its own stated limitations). v2 lets managers hire with money they were given, prices every cloud call from LiteLLM's price map [33] (local calls cost no dollars but hold the memory their model really uses, measured from Ollama [34]), and bounds every call by what its agent holds. Rules that v1 left to prompts (who may spend what, where private data may go) are enforced in code, because in our experience with an earlier prototype, rules stated only in prompts were broken.

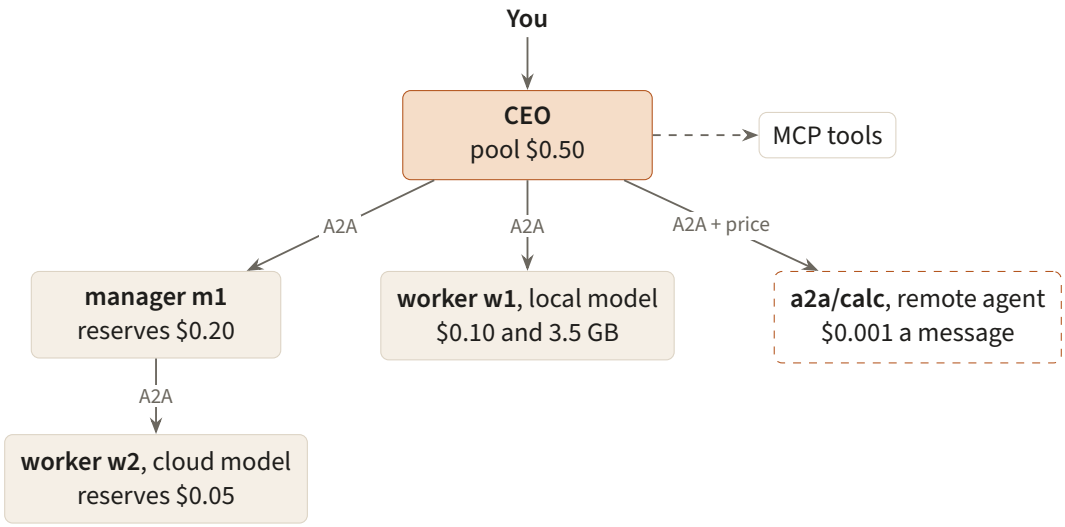


Figure 1: One hiring tree. Each hire reserves money from its hirer's pool (and a local model, the memory it uses); firing returns what it did not spend. A remote agent is paid its declared price from its hirer's reservation.

3.2 Spending bounded by construction

Each agent a holds a reservation R_a carved from its hirer's pool, and one rule says what it can still spend.

One rule for every pool

$$\text{left}(a) = R_a - \text{spent}(a) - \sum_{c \in \text{hires}(a)} \text{hold}(c) \tag{1}$$

A live hire holds its whole reservation; a fired one holds only what it (and everyone below it) actually spent.

Firing therefore returns the unspent part, and two hires running at once cannot spend the same dollars, since each holds its reservation from the moment it is hired. Table 2 works one session through.

Table 2: One session's pools (dollars). The CEO starts with $0.50.

Step	CEO left	m1 left
Session opens, `budget_usd` = 0.50	0.50	
CEO spends 0.01 thinking	0.49	
CEO hires manager m1 (0.20) and worker w1 (0.10)	0.19	0.20
m1 hires w2 (0.05) and spends 0.02	0.19	0.13
w1 spends 0.03, then the CEO fires it	0.26	0.13

A rule is only as good as its enforcement, and HASHIRU enforces this one in code at every call, not in the prompt. Three guards keep it true. (1) *Reservations.* A hire cannot reserve more than its hirer has left, a local hire must fit in free memory (or under a cap, `memory_mb`), and a remote agent's declared

price must fit in its reservation before each message. (2) *Bounded calls.* Before a cloud call, the hub prices its input and caps its output to what the agent can still pay for, so the call cannot cost more than the agent holds; a call that cannot pay for a short answer is refused before it reaches the provider, and its refusal states the price, so the hirer can reserve more. (3) *A closing turn.* An agent with one step left (its last allowed turn, or a pool that pays for an answer but not for another working turn and the answer after it) answers with what it has instead of ending with nothing. Together they hold each agent to what it holds, within the margin of the hub's token estimate, whatever the agent decides; §5 shows each one at work. Separately, `max_usd` caps real spend for the whole process, and because every reservation, call and fire is in the trace, the ledger can be rebuilt from the trace alone. Figure 2 follows one hire from its reservation to its refund.

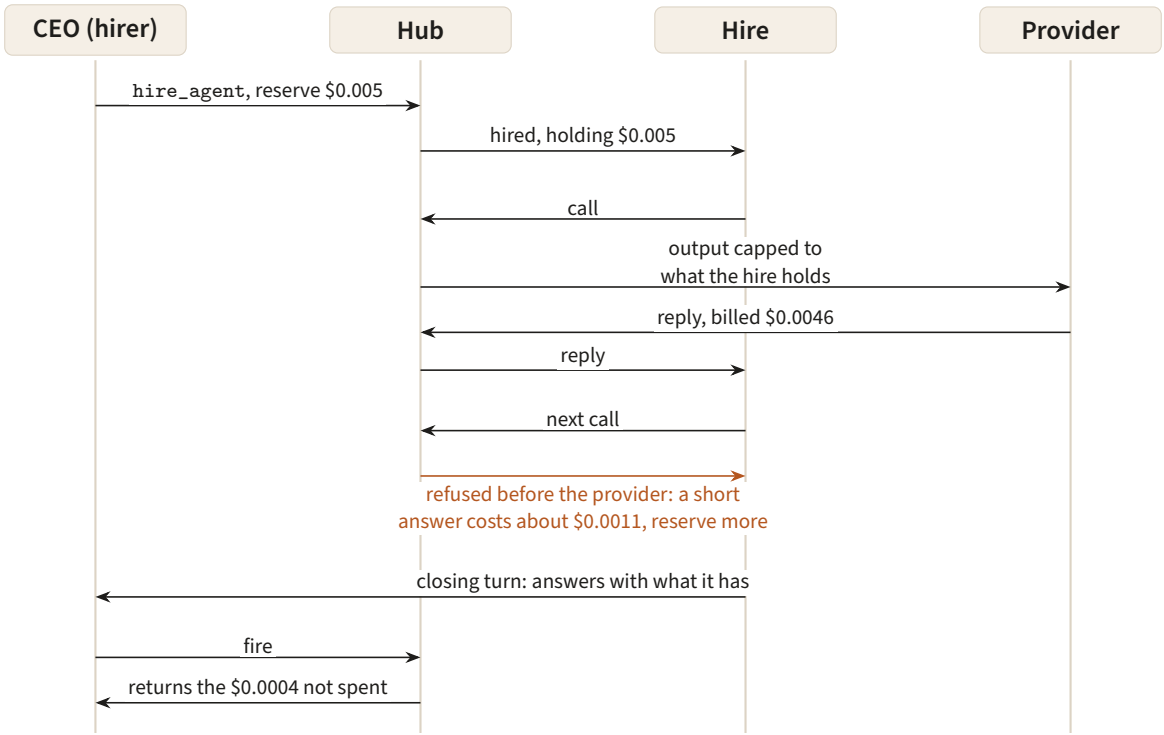


Figure 2: One hire's money, from reservation to refund, with the amounts of the median hire of `muse-spark-1.3` in §5. The hub reserves the hire's pool from its hirer's, caps each call's output to what the hire holds, refuses a call that cannot pay for a short answer before it reaches the provider (and quotes the price), and returns what was not spent when the hire is fired.

3.3 Hiring by declared difficulty

A hiring agent may name a model, or declare how hard a job is (easy, medium or hard) and let the hub choose. The hub turns a difficulty into a list of models, best first, and hires the first that the modes, the hirer's pool and the data's sensitivity allow. An easy job tries small local models, then cheap cloud models; a hard job tries the dearest cloud models first. A worker that fails or answers empty moves up its list and gets the message again, up to a limit (`max_escalations`, two by default). Which models exist, and their tiers, come from the machine itself (Ollama's pulled models that call tools, and every cloud model in LiteLLM's map whose provider key is set, tiered by price or by size).

Declared difficulty is the agent's own judgment, so PURSE measures it against the hidden difficulty of each job (§5).

A hirer may instead give the kind of job (`kind`) and let the hub route on measured success. Given a table of each model's success rate and mean cost per kind (`success_rates`, for instance from the hub's own traces), the hub hires the cheapest model that succeeds at least 70% of the time on that kind (`route_min_rate`), then the others, most reliable first. The hire's tool description lists the kinds the table knows, so the hirer can name one.

3.4 Make or buy over open protocols

Besides models, a manager may hire a remote A2A agent as a worker (`a2a/<name>`). The hub reads the agent's card, prices it through a small A2A extension (a card declares a price per message or per token; a reply reports what the request cost), charges the larger of the declared price and the reported cost to the hire's pool, and pins the agent to its configured URL. Since a HASHIRU hub is itself an A2A agent, one hub can hire another, and a whole team, with its own CEO and pools, appears to the hirer as one priced worker. This turns hiring into make or buy. An agent can make an answer with a model it pays per token, or buy it from an outside agent at a price. Either way, a hirer can attach sections of the user's request to its message verbatim, so a long input reaches the hire whole. With `request_by_reference` on, the CEO never reads a long section at all. It reads each section of 2,000 characters or more cut to its heading and first two lines, with a note saying how to pass it on, and a message to a hire that names a section by its key (`j3` for "### j3 (logic grids, ...)") carries the whole section. The CEO then pays to read the request's outline on every turn instead of the request itself.

3.5 Quotes, authority and sensitivity

Three further mechanisms read naturally in economic terms. *Plan mode is a quote.* The CEO may only read and propose; the hub previews each proposed hire as it would be made (the model the router picks, what it reserves, the memory it takes) and checks the totals against the pool, and approving the plan approves its hires. An approved plan's steps compile to a graph (LangGraph [35]) that hires, runs each step once its inputs exist, fires each hire after its last step, and lets the CEO answer once. *Approvals are spending authority.* A hire or a turn above a threshold waits for the user. *Sensitivity limits where work may go.* Private data never reaches a cloud or remote model as it is; it is masked with placeholders on the way out and restored on the way back (Presidio's recognizers [36] and a local model find the values), or refused.

3.6 Giving a team a budget

A team is one YAML file and one command (Figure 3). The file sets up the market of our evaluation, viz. the three cloud models, a priced logic-grid solver over A2A, and the second round's calibrated rates for level-5 arithmetic (Table 6). Every request then gets a CEO with a pool of `budget_usd`, the process stops at `max_usd`, and the trace records every reservation, call and refund.

3.7 Implementation

HASHIRU v2 is 6,578 lines of Python 3.13 with exact pins for its protocol and model libraries, and 625 tests that run without API keys or Ollama. The guards of §3.2 grew out of measurement, each written

```
# team.yaml
ceo_model: gpt-6-luna
models: [gpt-5-nano, gpt-6-luna, meta/muse-spark-1.3]
budget_usd: 0.10              # the CEO's pool for each request
max_usd: 5.00                 # real spend, for the whole process
request_by_reference: true    # long sections stay out of the CEO's context
a2a_peers:
  csp: {url: http://127.0.0.1:9512/}    # a priced logic-grid solver, hired as a2a/csp
success_rates:                # kind of job -> model -> [success rate, mean cost in $]
  arithmetic level 5: {gpt-6-luna: [0.25, 0.0031], gpt-5-nano: [0.53, 0.0073],
                       meta/muse-spark-1.3: [0.92, 0.0961]}

$ uv run --env-file .env hashiru ask --config team.yaml "Answer the jobs below ..."
```

Figure 3: A team with a budget: the market of our evaluation in one configuration file, and the command that sends it a request. Every key is one this section describes.

test-first after a benchmark run showed what happens without it (§5). So did routing on measured success, holding long sections back from the CEO, and four smaller features, viz. `attach`, which lets a hirer pass sections of the user's request to a hire word for word; a switch that takes the web tools away, so that no answer can be looked up during an evaluation; support for reasoning models that reject temperature 0; and a call refused for a provider's rate limit, which now waits and is sent again (for up to 104 seconds, unbilled) instead of reaching the agent as a failure.

4

How We Measured

To know whether agents spend well, we need tasks on which the best split of the money is known. Our evaluation suite, PURSE, builds them. Each project is a set of jobs that share one budget, every job has an exact answer, and every job has been tried on every option beforehand, so the best split for any budget can be computed. The score then moves with the split, and the gap to the best split says how much money an allocator left on the table.

4.1 Jobs and projects

A project is eight jobs, two from each of four families (Table 3), with one budget B. Each job has a visible level from 1 to 5 (its declared size parameters), a hidden difficulty bin (low, mid or high), a value of 1, 2 or 4 points drawn independently of difficulty, a canonical prompt and an exact checker. The allocator reads every job with its level and value and answers with one JSON object; the score is the share of value answered correctly. Jobs are generated from seeds, so they cannot be looked up, and HASHIRU's web tools are switched off for every run.

Table 3: PURSE's four job families.

Family	Level sets	Hidden bin sets	Can be bought
Arithmetic	operators (2 to 22), digits (2 to 8)	multiplications, digit sizes (carries)	`calc` (levels 1 to 4)
Logic grids	houses, attributes	case splits a propagating solver needs (0, 1, 2+)	`csp`; a free checker
Aggregation	records (40 to 2,500)	fields in the condition (1, 2, or 2 plus near misses)	no
String pipelines	operations (2 to 10), length	alphabet, position-dependent operations	no

For the second round of runs (§5) the bins were widened, so that knowing one would be worth more. An arithmetic bin also shifts the operands' digits (one fewer, as many, two more) and sets the share of operators that multiply (10%, 35% or 70%); an aggregation bin sets how many records match (a handful, about one in twenty, about one in seven) and triples the near misses; a string bin halves, keeps or doubles the start string, and in the high bin all but one operation depends on positions, while the low bin drops the Caesar shift.

4.2 Hidden difficulty and the observability dial

With probability ρ a job's hidden bin is the one its visible level suggests (levels 1 and 2 low, 3 mid, 4 and 5 high); otherwise it is drawn at random. At $\rho = 1$ the declared level is the whole story, so an up-front split with the right statistics loses nothing to an allocator that adapts; at $\rho = 0$ difficulty is learned by trying. The value of information, $\mathrm{VOI} = \mathrm{score}(\text{oracle knowing the bins}) - \mathrm{score}(\text{oracle knowing the levels})$, bounds what any adaptive allocator can gain over the best up-front split, and is reported for every condition. An allocator is given the statistics it would measure on projects of its own ρ, which are each bin's rates weighted by $P(\text{bin} \mid \text{level}) = \rho\,[\text{bin is the level's}] + (1-\rho)/3$.

4.3 The market

The options are model tiers at different prices (a cheap and a dear reasoning model in the first round, with a third, `gpt-5-nano`, in the second) and outside agents behind A2A. Two are exact programs, `calc` (exact arithmetic, HASHIRU's example agent, which reads expressions of up to 200 characters and so refuses level 5) and `csp` (a constraint solver for grids). The second round adds `gridfast`, which sells the same grids at a quarter of `csp`'s price and answers 40% of them wrong (one attribute exchanged between two houses), so that a cheap seller is not always a good one. Outside agents charge a price per message, swept over $c_{\mathrm{ref}} \times \{0.25, 1, 4, 16\}$, where c_{ref} is the cheap model's median cost on the jobs they can do. A free `grid_check` tool answers whether a grid solution satisfies every clue, at most three times per puzzle (checking is cheaper than solving, as unit tests are).

4.4 Calibration, replay and oracles

Every job is sent with its canonical prompt to every option that serves it (once in the first round, three times in the second), through a HASHIRU hub (a hire, one message, a fire), and the outcome table records each answer, whether it is right, and what it cost. A replayed call returns one of the job's

recorded attempts, and each seed starts from a different one. The table serves three purposes. It gives informed allocators their statistics, from the other half of the jobs (two folds), so a project never sees statistics that include its own jobs. It defines oracles. *Oracle-full* is a multiple-choice knapsack over each job's recorded success and cost (as in R^3-Bench [11]), an upper bound for every allocator. And it lets allocators that send canonical prompts be scored by replay on recorded outcomes instead of new calls, as RouterBench evaluates routers [21]. Budgets are set per project as $B = \tau\, C_{\text{full}}$, where C_{full} is what oracle-full spends at its best score with unlimited money; $\tau = 1$ is a control in which money does not bind.

4.5 Allocators

The main comparison crosses who allocates (a language model or a solver) with when (up front, or during the task), with the same project text, prices, statistics and budget for all four (Table 4).

Table 4: The four allocators in the main comparison.

	Up front	During the task
LLM	**LLM split.** The CEO plans one hire per job in plan mode, and the plan runs unchanged (ZEBRA's LLM baseline).	**HASHIRU.** The CEO hires, asks, fires, rehires and escalates as results arrive.
Solver	**Knapsack.** Maximizes expected points under the budget, from the statistics.	**Cascade.** Takes the cheapest option expected to succeed and moves up a tier on a failed check or two disagreeing answers.

Around them are oracle-full, fixed policies (every job on one model; a hybrid by visible level in AgentCARD's style [37]; an equal split; a random choice) and HASHIRU variants (the CEO naming models as in v1, and an uninformed CEO that sees prices but no statistics).

4.6 Validity checks

Before any allocator runs, seven checks computed from the outcome table hold PURSE to measuring allocation (Table 5). They are tested on synthetic tables built to pass, and on one table per check built to break exactly that check. Check V8 (replay agrees with live runs) needs live runs.

Both tables behind §5 fail three of the seven, and not the same three. On the first, V1 failed narrowly (oracle-full beat the best fixed policy by 9.7 points, not 10), V4 failed because the dear model left headroom in one family (long aggregation jobs), not two, and V5 failed outright (knowing each job's hidden bin was worth 0.009 of the value at $\rho = 0$, against the 0.08 the check asks for). The second table was built to fix them, with a third tier, a cheap and unreliable grid seller, wider bins and three attempts per job. V1 now holds (10.7 points). V5 still fails, and by more (0.002). The wider bins did separate the cheap tiers (`gpt-6-luna` answers 100% of low-bin aggregation jobs and 43% of mid-bin ones), but knowing the bins is worth between -0.007 and 0.002 of the value at every budget we tried, from $\tau = 0.25$ to 2, and 0.01 to 0.02 with the dear model taken out of the market. With four jobs a cell, a bin's rates rest on twelve attempts, which are as noisy as the differences they carry. V3 now fails because `gpt-5-nano` is not a weaker tier. It beats `gpt-6-luna` on arithmetic (53% against 25% at level 5) and falls behind it on aggregation and strings, so the three tiers cross by family instead of stacking, and on logic grids the dear model never leads the cheap one by 20 points. V4 fails as

before. On neither table, then, can the runs show what adapting during the task is worth, and the during-the-task column of Table 4 cannot be tested against the up-front one. A failed check means a generator is retuned and its cells calibrated again; we report both tables as they are, and a table that passes V5 is the first item on the roadmap (§7).

Table 5: Validity checks, from the outcome table, before any allocator runs, and what the two tables behind §5 give.

Check	Holds when	First table	Second table
V1 Allocation matters	oracle-full beats the best fixed policy by ≥ 10 points at $\tau = 0.5$	**fails** (9.7)	holds (10.7)
V2 The budget binds	the dear model alone covers $< 60\%$ of the value at $\tau = 0.5$	holds (59.6%)	holds (51.7%)
V3 Tiers separate	per family, a level where the cheap model succeeds $\geq 80\%$ (and a small tier, if present, $\leq 40\%$), and one where the dear model leads the cheap by ≥ 20 points	holds (4 of 4 families)	**fails** (0 of 4)
V4 Headroom	the dear model succeeds $\leq 60\%$ at level 5 in ≥ 2 families	**fails** (1 family)	**fails** (1 family)
V5 Observability	VOI ≤ 0.02 at $\rho = 1$ and ≥ 0.08 at $\rho = 0$	**fails** (0.000; 0.009)	**fails** (0.000; 0.002)
V6 Make or buy responds	oracle buying falls ≥ 40 points across the price sweep	holds (53 points)	holds (56 points)
V7 No shortcut	the most common answer per family scores $\leq 10\%$	holds (4.3%)	holds (5.6%)

5

Results

We ask whether agents that hold the money keep to it, whether they use the market that HASHIRU gives them, and how well they split the money.

5.1 Setup

We ran two rounds. The first (2026-11-01) built the guards and measured them; the second (2026-11-02) added a third model tier and an unreliable seller, recalibrated on a wider pool, and measured allocation on more projects.

Models and prices. The CEO is `gpt-6-luna` ($0.10 / $0.50 per million input / output tokens). The first round's market is `gpt-6-luna` (the cheap tier), `muse-spark-1.3` from Meta's Model API ($1.25 / $4.25, the dear tier), and the `calc` and `csp` agents. The second adds `gpt-5-nano` ($0.05 / $0.40) and `gridfast`. Prices are LiteLLM's map on the run dates. `gpt-5-nano` is the cheapest per token, but it reasons longer, and a calibration attempt on it costs three times what one on `gpt-6-luna` does ($0.0030 against $0.0010). Other small tiers were tried and set aside (Gemma 4 31B, whose hosted endpoint did not answer, and Muse Glimmer 30B, dearer per token than `gpt-6-luna`); local models

are left out, since on the laptop available (Apple M5, 24 GB) a 4B thinking model took six minutes on one arithmetic job.

Calibration. The first round sent 120 jobs (two per family, level and bin) once to each option, for $0.18 on `gpt-6-luna` and $3.18 on `muse-spark-1.3` (the outside agents are our own programs and cost nothing real). The second sent 240 jobs from the wider bins (four per cell) three times to each option, for $0.70 on `gpt-6-luna`, $2.17 on `gpt-5-nano` and $20.32 on `muse-spark-1.3`. Table 6 shows where the second round's tiers separate, and where they cross.

Table 6: Share of attempts answered correctly at the second round's calibration, by level (1 to 5; 36 attempts each).

Family	`gpt-5-nano`	`gpt-6-luna`	`muse-spark-1.3`	Outside agent
Arithmetic	100 100 100 81 **53**	94 100 97 **64 25**	100 100 100 92 92	100 100 100 100 **0** (`calc`)
Logic grids	94 100 100 100 94	100 94 94 94 94	100 100 100 100 100	100 at every level (`csp`) 67 67 50 58 50 (`gridfast`)
Aggregation	100 92 **44 25 0**	92 86 64 53 **36**	100 100 92 100 **47**	none
String pipelines	100 94 92 64 **47**	92 83 94 86 67	100 100 100 94 100	none

What these tables can show. Three of the seven validity checks fail on each table (§4.6), and each failure limits what this section can claim. Because V5 fails on both (knowing the hidden bins is worth at most 0.009 of the value at $\rho = 0$), these runs cannot show what adapting during the task is worth, for any allocator; the up-front and during-the-task allocators of Table 4 are compared here on what they score and spend, not on how well they adapt. Because V4 fails, the dear model solves nearly every job it is given (it falls to 60% or below only on level-5 aggregation), and because V3 fails on the second table, its three tiers do not stack, since `gpt-5-nano` beats `gpt-6-luna` on arithmetic and loses to it on aggregation and strings.

Runs. The first round drew ten projects at $\rho = 0.5$ and $\tau = 0.5$ (mean budget $0.056), and ran HASHIRU v2 live twice on each of the first six (mean budget $0.043). Its outside agents charged $0.001 a message live, against the c_{ref} of $0.00023 the replayed allocators paid. The second round draws twenty projects at the same ρ and τ (mean budget $0.058) and runs every allocator three times on each, HASHIRU v2 and the LLM split with three seeds and the replayed allocators on three of each job's recorded attempts. Its outside agents charge c_{ref} ($0.000215 a message, a quarter of that for `gridfast`) live and in replay alike. The solver allocators and fixed policies are replayed on the outcome table, and the LLM split's plan is made live and carried out by replay, each under two pool rules. Under the first hub's rule, a call needs money left and is charged in full, even past the pool; under v2's, a call is bounded by its pool before it is made, as it is for HASHIRU v2. Replay cannot know what a call cut at its pool would have answered, so it counts as no answer; all ten calls by hires that reached their pool's cap in the first round's live runs returned nothing. The first attempt at the second round's live runs sent nine processes at once, ran past the provider's limit of two million tokens a minute, and lost runs to calls refused for it; we discarded it and made a rate-limited call wait and retry (for up to 104 seconds), and no call in the runs we report failed for the limit. We discarded the second attempt as well, on finding that the project text put the market, the statistics and the answer format after the last job, inside that job's section, which `-by-reference` holds back from the CEO;

the text now puts them before the jobs, for every arm. All the money spent came to $33.21. The first round cost $4.50, and the second $23.42 to calibrate ($20.32 of it on `muse-spark-1.3`, and $0.23 on pilots), $4.30 for its live runs and $0.99 for the two discarded attempts.

5.2 What we found

Overall, our experiments show the following results:

The guards bound every hire. The second round made 120 live runs of HASHIRU v2 (two arms, twenty projects, three seeds) and 551 hires. No hire spent more than 1.11 times its reservation. Thirteen went over, by 9 to 11%, each on a single call, and all thirteen were aggregation jobs, whose inputs are long lists of records. A call's output is capped at what its pool has left after the estimated cost of its input, so an overrun can only come from the hub's token count running low on the input, beyond its 10% margin. In the first round, none of 48 hires went over (Table 7).

Each guard is necessary. In the first round we ran the same projects with HASHIRU as first built, without the call bound and the closing turn, and with the call bound alone (Table 7). Without the bound, a reservation was a ceiling checked before a call and the call's cost was charged after it. Hires that reserved $0.0005 spent up to 372 times that, the overruns came back to the CEO's pool, and three of six runs ended with no answer at a spend of 153% of the budget. With the bound alone, no hire overran, but the CEO rereads every job, long documents included, on every turn, and four of six runs ended without an answer (three when the CEO could not pay for its own answer, one at its 40-turn limit). With the closing turn as well, two of twelve runs were lost and the score rose to 0.566.

Table 7: HASHIRU with and without its guards, on the first round's six projects.

	As first built	Bounded calls only	HASHIRU v2
Runs	6	6	12
Mean score	0.374	0.278	**0.566**
Hires that spent past their reservation	9 of 26	0 of 25	**0 of 48**
Largest overrun (spent / reserved)	372×	1.0×	**1.0×**
Runs that ended without an answer	3 of 6	4 of 6	**2 of 12**
Spent / budget	1.53	0.62	**0.71**
Calls refused with a quote	0	19	25

Keeping long inputs out of the CEO's context raises the agents' score by 0.136. The second round ran HASHIRU v2 as it was and with two changes, long sections held back from the CEO (`request_by_reference`) and hires routed on the success rates the CEO is shown (`success_rates`). With both, the agents scored 0.599 of the value against 0.463, a paired difference of $+0.136$ over the twenty projects (95% interval $[+0.067, +0.216]$). The CEO reads the request's outline (8.7 thousand characters on the first project) instead of the request (264 thousand), and its own calls fell from 43% of the budget to 32%. Messages to outside agents now carry the puzzle the CEO names, so `csp` answered 48 of its 49 messages right, where 24 of its 44 replies had been "no grid puzzle in the text", and the outside agents together answered 74 of 119 right, against 30 of 122. Runs in which the CEO could not pay to read its request once fell from 12 to 0, and lost runs from 17 to 11. We did not run the two changes apart; every mechanism above is one of holding sections back, and what routing on rates adds alone is not measured.

On projects whose budget pays for the CEO, the agents match the cheap model alone. Table 8 holds every allocator to v2's pool rule, with the first rule's scores beside them. On the sixteen projects whose budget is $0.006 or more, the agents scored 0.737, level with `gpt-6-luna` alone (0.740; -0.003, $[-0.038, +0.030]$) and the equal split (0.761; -0.024, $[-0.056, +0.007]$), above random choice ($+0.067$, $[+0.005, +0.134]$), and above the cascade, the LLM split and `gpt-5-nano` alone by margins that sixteen projects do not resolve. They trail the knapsack (-0.080, $[-0.129, -0.023]$), and they spent 3.9 times what the cheap model alone spent.

The agents win on hard jobs that a specialist can do. Two families have an outside agent in the market (`calc` for arithmetic, `csp` and `gridfast` for logic grids). On the sixteen projects, over the jobs of those families whose hidden difficulty is high (26 jobs, three runs each), the agents answered 62 of 78 right (79%) against 54 of 78 for `gpt-6-luna` alone (69%), 24 against 18 of the hard arithmetic and 38 against 36 of the hard grids. They were ahead on six projects and behind on two, an interval over projects that just reaches zero. They send grids to `csp`, which answered all 41 of its messages on these projects right, and they give hard jobs a second hire more often than easy ones (46 of the 81 jobs that went to more than one hire were hard, where hard jobs are 37% of all jobs). On the other jobs they trail the cheap model (226 of 306 right, 74%, against 239, 78%), and the two cancel to the level score above.

Below a budget floor, the agents cannot work. Four of the twenty projects have budgets of $0.0009 to $0.0051, since their jobs are cheap to solve and $B = \tau\, C_{\text{full}}$. On them `gpt-6-luna` alone scores 0.724, and the LLM split, whose one planning call is charged to the budget, scores 0.635. The agents score 0.046. A CEO that hires, asks and fires pays for a turn at each step, and all eleven of the runs the agents lost were on these four projects, where the CEO came back empty after spending most or all of the budget on its own calls; on the other sixteen they lost none of 48. Over all twenty projects the agents' 0.599 trails the cheap model alone (0.736).

Table 8: Allocators on the second round's twenty projects ($\rho = 0.5$, $\tau = 0.5$, mean budget $0.058), ordered by score under v2's pool rule, and on the sixteen whose budget is $0.006 or more. Each is run three times per project (sixty runs). "Spend vs. Luna" is spend over that of every job on `gpt-6-luna` ($0.0079 a project). The last column replays the same allocators under the first hub's rule, which charges a call in full even past the pool. HASHIRU v2's two arms run live, as it was and holding long sections back (during the task).

Allocator	Score	Sixteen	Spent ($)	Of budget	Vs. Luna	Own calls ($)	First rule
Oracle-full (knows outcomes)	0.826	0.838	0.028	0.48	3.50×	0	0.830 (0.49)
Knapsack (solver, up front)	0.788	0.817	0.035	0.60	4.42×	0	0.794 (0.68)
Every job on `gpt-6-luna`	0.736	0.740	0.008	0.14	1.00×	0	0.723 (0.15)
Equal split	0.713	0.761	0.011	0.18	1.35×	0	0.713 (0.18)
LLM split (LLM, up front)	0.695	0.709	0.033	0.57	4.15×	0.006	0.699 (0.69)
Cascade (solver, during)	0.682	0.712	0.050	0.86	6.29×	0	0.762 (1.58)
Every job on `gpt-5-nano`	0.628	0.700	0.023	0.39	2.87×	0	0.630 (0.41)
Random	0.612	0.670	0.029	0.50	3.69×	0	0.614 (1.05)
HASHIRU v2 by reference (agents)	**0.599**	**0.737**	**0.031**	**0.52**	**3.84×**	0.006	live
Hybrid by level	0.493	0.494	0.052	0.90	6.60×	0	0.531 (2.14)
HASHIRU v2 as it was (agents)	0.463	0.579	0.036	0.61	4.47×	0.025	live
Every job on `muse-spark-1.3`	0.342	0.427	0.054	0.93	6.82×	0	0.447 (2.04)

First rule: score, with spent / budget in parentheses. "Sixteen": score on the sixteen projects whose budget is $0.006 or more.

The agents respond to prices. A hire's call that its pool cannot pay for is refused before it reaches the provider, and the refusal quotes the price. In the second round, 203 calls were refused this way in the arm that holds sections back (253 in the other), and in 112 of them a later hire in the same run reserved more than the refused one had (156). Table 9 follows one such exchange from a first-round trace. The CEO declared a level-4 logic grid easy and gave it to `gpt-6-luna` with $0.001. The hire's first call reasoned up to the cap its pool allowed and returned nothing, its next call was refused, and so was the same call on the dear model after the hub escalated it. The CEO then reserved five times as much for a new hire, and twelve times as much for a third, whose answer was right and worth 4 of the project's 22 points.

Table 9: A quoted refusal and what followed, from the trace of one first-round live run (project p2, second run; job j4, a level-4 logic grid worth 4 of the project's 22 points).

	Reserved	What happened
1	$0.001	The CEO declares the job easy, and the hub hires `grid4` on `gpt-6-luna`.
2		`grid4`'s first call reasons up to the cap its pool allows, spends $0.0010 and returns nothing.
3		Its next call is refused before it reaches the provider.
4		The hub escalates `grid4` to `muse-spark-1.3`, and the refusal quotes the price: "grid4's pool has $0.0000, and a call on meta/muse-spark-1.3 costs about $0.0005 to read its messages and $0.0011 for a short answer: reserve more."
5	$0.005	The CEO checks its own pool and hires `grid4b`, which answers that the clues are inconsistent (wrong).
6	$0.012	The CEO hires `solve4`, whose solution is right.

The agents choose the dear model for the right jobs, but fund it below its price. On the sixteen projects, 65% of the agents' tries on `muse-spark-1.3` (68 of 104) were on hard jobs, and half came after a cheaper try on the same job. The 79 hires made on it took 55% of what the agents spent ($0.99 of $1.81) and returned 12 right answers, where its calibrated rate on hard jobs is 87 to 100%. The median hire reserved $0.005, while an attempt on the dear model costs a median of $0.006 to $0.025 depending on the family (a mean of $0.010 to $0.061). So 78 of the 79 hires had a call refused because their pool could not pay for it, and they had spent nearly all of their reservation first (a median of $0.0046). The quoted refusal arrives after the money is gone, and at $\tau = 0.5$ the budget pays for a full attempt on the dear model on only a few jobs, so these hires buy calls that cannot finish.

Allocating pays once the budget has room, and the cheap model alone cannot use it. The live runs hold $\tau = 0.5$. Replaying the outcome table at τ of 0.25, 1 and 2 (no model calls, Figure 4 and Table 10) shows where an allocator can win. On the sixteen projects, `gpt-6-luna` alone scores 0.740 to 0.749 whatever the budget, since it spends 31% of the budget at $\tau = 0.25$ and 7% at $\tau = 2$ and has nothing to buy with the rest. The cascade adapts during the task as the agents do (it tries the cheapest option expected to be good enough, moves up when a grid's check fails, and asks a third option when the two cheapest disagree). It trails the cheap model by 0.051 at $\tau = 0.25$ and leads it by 0.133 at $\tau = 2$ ($[+0.078, +0.192]$, ahead on 12 of 16 projects), and the knapsack's lead grows from 0.029 to 0.162. Which single model is best changes as well, since the dear model alone passes the cheap one at $\tau = 2$ (0.815 against 0.749). The knapsack, which plans up front, leads by more than the cascade at every budget, so the sweep does not say that adapting beats planning (V5 fails, as above). It says that a budget with room rewards an allocator that uses it. The agents ran only at $\tau = 0.5$, where the cascade still trails the cheap model and the agents' tries on the dear model go unfunded. We did

not run them at a larger budget, so whether they follow the cascade there is not measured, but the sweep says that a larger budget is where they should win.

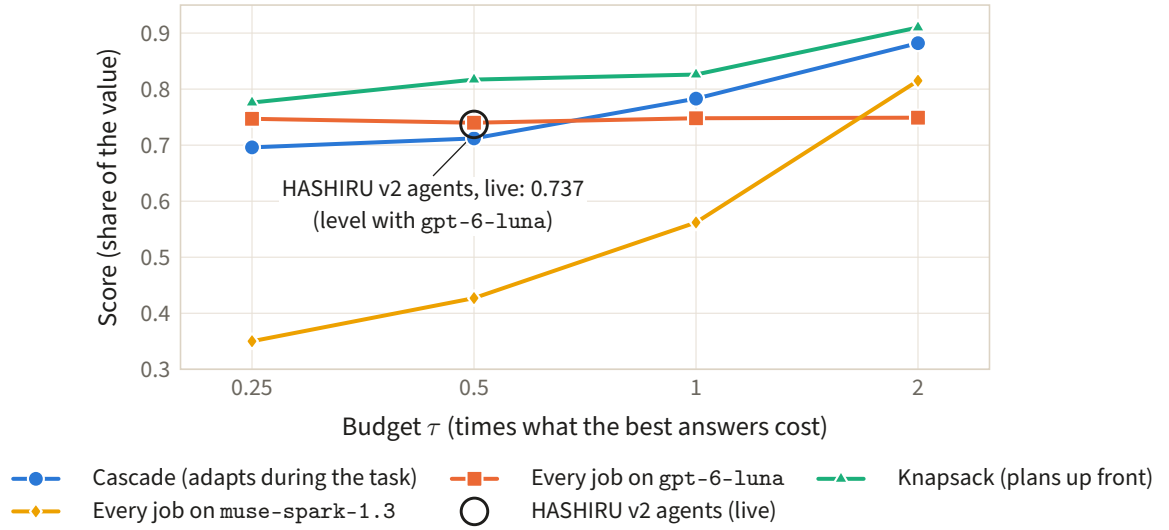


Figure 4: Where allocating pays. Scores on the sixteen projects as the budget grows eightfold, replayed from the outcome table (three runs per project). The cheap model alone stays flat; the cascade and the knapsack turn the extra budget into score. The agents ran live at $\tau = 0.5$ only (ringed). Table 10 gives the numbers and intervals.

Table 10: Replayed allocators on the sixteen projects at four budgets ($B = \tau\, C_{\text{full}}$, a mean of \$0.036, \$0.072, \$0.144 and \$0.288 a project), three runs each under v2's pool rule. The agents ran live at $\tau = 0.5$ only. The last row is the cascade's paired lead over the cheap model alone, with its 95% interval over projects.

Allocator	$\tau = 0.25$	$\tau = 0.5$	$\tau = 1$	$\tau = 2$
Every job on `gpt-6-luna`	0.747	0.740	0.748	0.749
Every job on `gpt-5-nano`	0.635	0.700	0.721	0.723
Every job on `muse-spark-1.3`	0.350	0.427	0.562	0.815
Equal split	0.748	0.761	0.791	0.808
Cascade (solver, during)	0.696	0.712	0.783	**0.882**
Knapsack (solver, up front)	0.776	0.817	0.826	0.910
Oracle-full (knows outcomes)	0.825	0.838	0.844	0.947
HASHIRU v2 by reference (agents, live)		0.737		
Cascade minus `gpt-6-luna`	-0.051 $[-0.108, +0.004]$	-0.028 $[-0.088, +0.031]$	$+0.035$ $[-0.036, +0.100]$	$\mathbf{+0.133}$ $[+0.078, +0.192]$

What the agents still get wrong. Beyond the dear model, two things separate them from the knapsack. First, the CEO keeps jobs for itself and answers them worse than a fresh call does. In the arm that holds sections back, it answered 169 jobs itself and got 85% of them right, where `gpt-6-luna` given each job alone gets 91%; the 223 jobs it handed to hires were right 67% of the time, against 68%. Second, the difficulty it declares does not track the jobs' hidden difficulty (rank correlation 0.10 over 128 hires), and its own calls still take 32% of the budget. It rarely buys from the cheap, unreliable `gridfast` (seven messages, five of them right).

Holding the baselines to v2's rule lowers the allocators that spent past their budget under the first rule. The cascade, the hybrid by level and the dear model alone, which spent 1.58 to 2.14 times their budget under the first rule, fell by 0.080, 0.038 and 0.105; random choice and the knapsack fell by only 0.002 and 0.006. It raised the cheap model alone by 0.013, since refusing a call its pool cannot pay for leaves the money for the jobs after it (on one small project, the first rule let the cheap model's calls spend up to 2.9 times the budget, and it scored 0.4 to 0.6 less).

6

Limitations and Scope

Scale of the runs. The second round has twenty projects, three runs of each allocator on each, and three calibration attempts per job and option, so replay carries some of the models' own variance; the intervals we give are over projects, and with twenty (or sixteen) they are still wide. Both outcome tables fail three of their seven validity checks (§4.6), V5 among them, so neither round can show what adapting during the task is worth.

Live against replay. The baselines are replayed with each job's canonical prompt, while HASHIRU v2 ran live and wrote its own messages to its hires. Replay counts a call cut at its pool as no answer, which the first round's live runs bear out (all ten such calls by hires returned nothing). In the first round the outside agents charged HASHIRU $0.001 a message live, about four times the c_{ref} the replayed allocators paid; in the second, live and replay paid the same price. The budget sweep (Table 10) is replay only, since the agents ran live at one budget, so it shows where an allocator can win, not that the agents do.

What was varied. The second round changed two things at once (holding long sections back and routing on measured rates) and did not run them apart. The traces tie each measured effect to holding sections back, but what routing on rates adds alone is not measured. Its rates are the same statistics the CEO is shown, so it cannot know more than the CEO is told.

Small budgets. Budgets follow $B = \tau\, C_{\mathrm{full}}$, so a project of cheap jobs gets a small budget, and four of the second round's twenty have less than $0.006. An allocator that pays for its own turns cannot work below that, and we report those projects apart rather than leave them out.

The market and the system. Prices are list prices on the run dates, and the runs have three cloud tiers and no local model, so memory as a second currency is tested in HASHIRU but not measured here. Declared difficulty is the agent's own judgment. A call is bounded by the hub's token count, which ran low by up to 11% of a reservation on long aggregation inputs (13 of 551 hires). One API key is one trust domain.

7

Toward Economies of Agents

HASHIRU v2 shows that language-model agents can be handed money and held to it. Each agent holds a pool reserved from its hirer's, every call is bounded by what its agent holds, firing returns what was not spent, and an agent with one step left is made to answer. Over 551 hires in 120 live runs, no hire spent more than 1.11 times its reservation, where HASHIRU as first built let one spend 372 times. Holding long inputs out of the CEO's context raised the agents' score by 0.136. On the projects whose budget pays for the CEO, the agents beat the cheap model alone on hard jobs that a specialist can do (79% against 69% right) and score level with it overall (0.737 against 0.740), at 3.9 times its spend. Because every reservation, call and refund is in the trace, the spending can be checked, measured and improved, which is how the guards and both changes came about and how we located what remains.

When to use HASHIRU v2 today

Our runs support five rules of thumb, each scoped to the market we measured.

- **A hard spending limit.** Use HASHIRU when a team must stay inside its money. Every hire stayed within 1.11 times its reservation, where the same agents without the call bound overran one by 372 times.
- **Hard jobs that a specialist can do.** Hire through HASHIRU when the market sells an exact answer. On hard arithmetic and logic grids, the agents were right 79% of the time, against 69% for the cheap model alone.
- **A budget that cannot pay for the dear model.** Expect the cheap model's score at a higher spend. At the budgets we ran, the agents scored level with `gpt-6-luna` alone (0.737 against 0.740) at 3.9 times its spend.
- **A very small budget.** Below about $0.006 a project, send each job to one cheap model. A CEO that pays for each turn spent the budget before it answered (0.046 against 0.724).
- **A budget with room.** Allocate rather than fix one model. In replay, the cheap model alone stayed near 0.75 as the budget grew eightfold while the cascade reached 0.88, though the agents have not yet been run there.

This is a step toward agent economies in which teams of agents hold budgets, buy from one another and account for every dollar. The pieces are already in HASHIRU. Priced remote agents and whole remote teams are hired like any model, prices travel in an A2A extension, and payment protocols (AP2 [29], the A2A x402 extension [30]) can settle what the ledger records. An organization could give a team of agents a budget the way it gives one to a department, and get back an answer, a spend within that budget, and a trace of where each dollar went.

Roadmap. Each remaining gap has a concrete next step, and we list them by payoff per dollar. The agents pick the dear model for hard jobs but fund it below its price, so more than half of what they spent went to its hires, and 12 of 79 answered right. The hub knows what an attempt on each model costs, from the same statistics it routes on, so it can quote that price when the hire is made rather than after the money is spent, and the CEO can then fund the hire in full or buy something cheaper. With that in place, the agents should be run at larger budgets, where the replays show an allocator pulling ahead of the cheap model (0.133 ahead at twice the cost of the best answers). The agents cannot work below a budget floor, since a CEO that hires, asks and fires pays for a turn at each step; below it, the hub can plan up front in one call, as the LLM split does, or answer cheap projects with one model. The CEO answers the jobs it keeps worse than a fresh call (85% against 91%), which argues for delegating by default. The call bound is only as tight as the hub's token count, which can be checked against billed usage in the trace and its margin learned from it. The two changes of the second round should be run apart. And a table that passes V5 needs more jobs per cell than four, since a bin's rate from twelve attempts is as noisy as the differences the bins carry; without it, the value of adapting during the task cannot be measured. Finally, the market can grow to local models, with memory as a second currency, which HASHIRU already supports.

Availability. HASHIRU v2 is source-available under the PolyForm Noncommercial License 1.0.0 at https://github.com/HASHIRU-AI/HASHIRU. The evaluation suite and the run data are available from the authors on request. HASHIRU v1's code is preserved at tag `v1.0-paper`.

Acknowledgements

We thank Parth Shah for building much of HASHIRU v1, on which this work builds. We thank Hugging Face, Groq and Lambda Labs for sponsoring HASHIRU and providing the resources for its development, and Y Combinator's Startup School for API credits.

References

[1] Kunal Pai, Parth Shah, and Harshil Patel. HASHIRU: Hierarchical agent system for hybrid intelligent resource utilization, 2025. URL https://arxiv.org/abs/2506.04255.

[2] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=BAakY1hNKS.

[3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VtmBAGCN7o.

[4] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-One: A generalist multi-agent system for solving complex tasks, 2024. URL https://arxiv.org/abs/2411.04468.

[5] Conway Research. Automaton. https://github.com/Conway-Research/automaton, 2026. Agent runtime; the `fund_child` tool transfers credits to a child agent. Accessed 2026-11-01.

[6] FerroxLabs. wayland-core. https://github.com/FerroxLabs/wayland-core, 2026. Agent runtime; the `delegate` tool gives a sub-agent a USD ceiling. Accessed 2026-11-01.

[7] Xiao Liu, Haoyang Li, Songwei Li, Hongbo Fang, Fengli Xu, Feng Shi, and James Evans. Markets, not planners: Decentralized orchestration of llm agents with private information, 2026. URL https://arxiv.org/abs/2608.23867.

[8] Liming Yang, Junyu Luo, Xuanzhe Liu, Yiling Lou, and Zhenpeng Chen. BAMAS: Structuring budget-aware multi-agent systems. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(35):29802–29810, 2026. doi: $10.1609/\mathrm{aaai.v40i35.40226}$. URL https://doi.org/10.1609/aaai.v40i35.40226.

[9] May Hamri and Inbal Talgam-Cohen. ZEBRA: Zero-shot budgeted resource allocation for LLM orchestration, 2026. URL https://arxiv.org/abs/2605.20485.

[10] Yuxiang Lin, Zihan Wang, Mengyang Liu, Yuxuan Shan, Longju Bai, Junyao Zhang, Xing Jin, Boshan Chen, Jinyan Su, Xingyao Wang, Jiaxin Pei, and Manling Li. BAGEN: Are LLM agents budget-aware?, 2026. URL https://arxiv.org/abs/2606.00198.

[11] Peisong Wang, Zhiwei Ma, Bowen Liu, Feixue Liu, Aochuan Chen, Chenyi Zi, Hongchuan Zeng, Yuhan Li, and Jia Li. r^3-bench: Llms struggle with resource-rational reasoning under shared budgets, 2026. URL https://arxiv.org/abs/2608.16033.

[12] Jie Wu, Ming Gong, Feixiang Cheng, and Qinqin Zhao. Ecoagent-bench: Evaluating economic decision-making in budget-constrained llm agents, 2026. URL https://arxiv.org/abs/2608.05519.

[13] Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyan Qi. MasRouter: Learning to route LLMs for multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15549–15572, Vienna, Austria, 2025. Association for Computational Linguistics. doi: $10.18653/\mathrm{v1}/2025.\mathrm{acl\text{-}long}.757$. URL https://aclanthology.org/2025.acl-long.757/.

[14] Xinkui Zhao, Sai Liu, Yifan Zhang, Qingyu Ma, Zewen Lin, Naibo Wang, Guanjie Cheng, Chang Liu, and Yueshen Xu. ATOM: Instantiating budget-controllable multi-agent collaboration via nucleus-electron hierarchy, 2026. URL https://arxiv.org/abs/2605.26178.

[15] Antonio Sabbatella. MALBO: Optimizing LLM-based multi-agent teams via multi-objective Bayesian optimization, 2025. URL https://arxiv.org/abs/2511.11788. Master's thesis, University of Milano-Bicocca.

[16] Di Zhao, Longhui Ma, Siwei Wang, Miao Wang, and Yi Kong. SC-MAS: Constructing cost-efficient multi-agent systems with edge-level heterogeneous collaboration, 2026. URL https://arxiv.org/abs/2601.09434.

[17] Tims Pecerskis and Aivars Smirnovs. Mixture-of-Models: Unifying heterogeneous agents via N-way self-evaluating deliberation, 2026. URL https://arxiv.org/abs/2601.16863.

[18] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs from preference data. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=8sSqNntaMr.

[19] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=02f3mUtqnM.

[20] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=cSimKw5p6R.

[21] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. Routerbench: A benchmark for multi-llm routing system, 2024. URL https://arxiv.org/abs/2403.12031.

[22] Tengxiao Liu, Zifeng Wang, Jin Miao, I-Hung Hsu, Jun Yan, Jiefeng Chen, Rujun Han, Fangyuan Xu, Yanfei Chen, Ke Jiang, Samira Daruki, Yi Liang, William Yang Wang, Tomas Pfister, and Chen-Yu Lee. Budget-aware tool use enables effective agent scaling. In *Third Conference on Language Modeling*, 2026. URL https://openreview.net/forum?id=hZ5XPW7jaD.

[23] Daniel Wang and Andrew Xu. Allocbench: Measuring online tool allocation capability in llm agents, 2026. URL https://arxiv.org/abs/2607.23332.

[24] Jiayu Liu, Cheng Qian, Zhaochen Su, Qing Zong, Shijue Huang, Bingxiang He, and Yi R. Fung. CostBench: Evaluating multi-turn cost-optimal planning and adaptation in dynamic environments for LLM tool-use agents. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12826–12858, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.584. URL https://aclanthology.org/2026.acl-long.584/.

[25] Avanika Narayan, Dan Biderman, Sabri Eyuboglu, Avner May, Scott Linderman, James Zou, and Christopher Re. Cost-efficient collaboration between on-device and cloud language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 45682–45719. PMLR, 2025. URL https://proceedings.mlr.press/v267/narayan25a.html.

[26] Siyan Li, Vethavikashini Chithrra Raghuram, Omar Khattab, Julia Hirschberg, and Zhou Yu. PAPILLON: Privacy preservation from Internet-based and local language model ensembles. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3371–3390, Albuquerque, New Mexico, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.naacl-long.173. URL https://aclanthology.org/2025.naacl-long.173/.

[27] Nenad Tomasev, Matija Franklin, Joel Z. Leibo, Julian Jacobs, William A. Cunningham, Iason Gabriel, and Simon Osindero. Virtual agent economies, 2025. URL https://arxiv.org/abs/2509.10147.

[28] Gagan Bansal, Wenyue Hua, Zezhou Huang, Adam Fourney, Amanda Swearngin, Will Epperson, Tyler Payne, Jake M. Hofman, Brendan Lucier, Chinmay Singh, Markus Mobius, Akshay Nambi, Archana Yadav, Kevin Gao, David M. Rothschild, Aleksandrs Slivkins, Daniel G. Goldstein, Hussein Mozannar, Nicole Immorlica, Maya Murad, Matthew Vogel, Subbarao Kambhampati, Eric Horvitz, and Saleema Amershi. Magentic Marketplace: An open-source environment for studying agentic markets, 2025. URL https://arxiv.org/abs/2510.25779.

[29] Google. Agent Payments Protocol (AP2) specification, v0.2, 2026. URL https://ap2-protocol.org/ap2/specification/. Accessed 2026-11-01.

[30] Google Agentic Commerce. A2A protocol: x402 payments extension v0.2, 2025. URL https://github.com/google-agentic-commerce/a2a-x402/blob/main/spec/v0.2/spec.md. Accessed 2026-11-01.

[31] Model Context Protocol, a Series of LF Projects, LLC. Model Context Protocol specification, version 2026-07-28, 2026. URL https://modelcontextprotocol.io/specification/2026-07-28. Accessed 2026-11-01.

[32] Agent2Agent (A2A) Project. Agent2Agent (A2A) protocol specification, version 1.0.0, 2026. URL https://a2a-protocol.org/v1.0.0/specification/. Accessed 2026-11-01.

[33] Berri AI. LiteLLM, 2026. URL https://github.com/BerriAI/litellm. Software, version 1.106.6. Accessed 2026-11-01.

[34] Ollama. Ollama, 2026. URL https://github.com/ollama/ollama. Software, version 0.40.13. Accessed 2026-11-01.

[35] LangChain. LangGraph, 2026. URL https://github.com/langchain-ai/langgraph. Software, version 1.2.19. Accessed 2026-11-01.

[36] Presidio Contributors. Presidio: Data protection and de-identification SDK, 2026. URL https://github.com/data-privacy-stack/presidio. Software, version 2.2.366. Originally developed by Microsoft; now maintained by the Data Privacy Stack community. Accessed 2026-11-01.

[37] Yinsicheng Jiang, Liang Cheng, Yeqi Huang, Yufan Zhao, Zhan Lu, Li Dong, Wenda Li, Edoardo Ponti, and Luo Mai. Specialize roles, mix deployments: Pushing the cost-accuracy frontier of LLM agent teams, 2026. URL https://arxiv.org/abs/2606.20629.